

Отправка данных в строке запроса

- Отправка простых переменных
- Отправка одномерных массивов
- Отправка многомерных массивов

Отправка простых переменных

Самым простым и часто применяемым способом передачи данных сценарию PHP является **передача данных через строку запроса**.

Строка запроса представляет собой **набор параметров**, которые помещаются в адресе **после вопросительного знака**. При этом каждый параметр определяет **имя** и **значение**. Например, в адресе:

```
http://php-course/example_1.php?user=gyrgen&topic=5&lesson=3
```

часть:

```
?user=gyrgen&topic=5&lesson=3
```

представляет **строку запроса**, в которой есть три **параметра**. Для каждого параметра определены **имя** и **значение**, которые отделяются знаком равно:

- **user** = gyrgen
- **topic** = 5
- **lesson** = 3

Друг от друга параметры отделяются знаком амперсанда (&).

Когда мы вводим в адресную строку браузера некий адрес и отправляем запрос, то **серверу отправляется запрос типа GET**.

В PHP по умолчанию определен **суперглобальный ассоциативный массив `$_GET`**, который хранит все значения, **передаваемые в запросе GET**.

Используя имена параметров в качестве **ключей** ассоциативного массива **`$_GET`**, можно получить из строки запроса **передаваемые значения**.

Рассмотрим скрипт, извлекающий из строки запроса простые переменные и передающий эти переменные на вывод в браузер.

example_1. Отправка простых переменных

```
<?php

    // исходная адресная строка имеет вид:
    // http://php-course/example_1.php

    // добавить к адресной строке параметры запроса
    // ?user=gyrgen&topic=5&lesson=3

    // или перейти по ссылке

    echo "<a href='http://php-course/example_1.php
    ?user=gyrgen&topic=5&lesson=3'>

    Для формирования GET-параметров кликните по ссылке

    </a>";

    if (isset($_GET['user']) && isset($_GET['topic']) &&
        isset($_GET['lesson'])) {

        echo "

            <h3>Пользователь {$_GET['user']} запросил
            материал:</h3>

            Тема: {$_GET['topic']}<br />

            Урок: {$_GET['lesson']}

            ";

    }
```

```
else

echo "<h3>Не указан требуемый набор GET-параметров.</h3>";

?>
```

Важно. При отправке строки запроса:

- **имена параметров** станут **ключами** глобального массива `$_GET`
- **значения параметров** станут **значениями** массива.

Например, в строке запроса передается параметр `name=Tom`.

Соответственно, чтобы получить значение параметра `name` из строки запроса, обращаемся к массиву **`$_GET`** по соответствующему ключу:

```
$name = $_GET["name"]; // Tom
```

Однако, стоит учитывать, адресную строку легко модифицировать и в ней вполне может не оказаться ожидаемой **строки запроса** или **конкретного параметра**. Например, адресную строку:

```
http://php-course/example_1.php?user=gyrgen&topic=5&lesson=3
```

пользователь может легко модифицировать в:

```
http://php-course/example_1.php?topic=5&lesson=3
```

или даже в:

```
http://php-course/example_1.php
```

Поэтому перед **получением значения параметра** сначала стоит проверить, а существует ли вообще такой параметр.

В демонстрационном примере **example_2** самостоятельно добавьте к **адресной строке** предлагаемые параметры запроса.

example_2. Проверка параметра на его наличие

```
<?php

// исходная адресная строка имеет вид:
// http://php-course/example_2.php
// самостоятельно добавьте к адресной строке параметры
// запроса

// вариант 1. ?user=gyrgen&topic=5&lesson=3
// вариант 2. ?topic=5&lesson=3
// вариант 3. ?user=gyrgen&topic=5
// вариант 4. ?user=gyrgen&lesson=3

if (isset($_GET["user"])){

    $user = "Пользователь {$_GET['user']} запросил
    материал:";

}

    else $user = "Пользователь не выбран.";

if (isset($_GET["topic"])){

    $topic = $_GET['topic'];

}

    else $topic = "Нет информации";

if (isset($_GET["lesson"])){

    $lesson = $_GET['lesson'];

}

    else $lesson = "Нет информации";

echo "

    <h3>$user</h3>

    Тема: $topic<br />

    Урок: $lesson
```

```
" ;  
?>
```

Важно. Объем данных, пересылаемых в адресной строке с помощью GET-запроса, зависит от **сервера** и **типа браузера**. Средний размер, на который можно ориентироваться - 4 Кб. Таким образом, начинающий разработчик может смело пересылать свои 100-1000 символов.

Отправка одномерных массивов

В прошлом разделе была рассмотрена отправка на сервер простых значений. Рассмотрим, как можно **отправить** и соответственно **получить** на сервере структурированные **массивы данных**.

Примечание. В этом разделе мы поговорим о способах формирования строки GET-параметров. Но самое главное, я подведу вас к такой важной теме как **JSON-представление** данных.

Суть передачи массива в строке запроса это предварительное приведение данного массива к **строковому представлению** (там, где это возможно).

В качестве примера иницилируем и приведем к строковому представлению **одномерный индексный массив**.

example_3. Передача одномерного индексного массива

```
<?php  
  
// исходный адрес скрипта имеет вид:  
  
// http://php-course/example_3.php
```

```
// инициализируем одномерный индексный массив

$data = ["Toyota", "Camry", "2.5"];

// преобразуем массив к строке GET-параметров

// добавим строку параметров к адресу скрипта

// ?data[]=Toyota&data[]=Camry&data[]=2.5

// для проверки наличия GET-параметра используем краткую
форму тернарного оператора

$_GET["data"] = $_GET["data"] ?? "Нет GET-параметров";

echo "<pre>";

print_r($_GET["data"]);

echo "</pre>";

?>
```

Строковое представление массива, выполненное вручную, может быть неплохим **развлечением** в свободное от программирования время. Но формирование таким способом строк запроса в реальных проектах это утопия. Процесс необходимо автоматизировать.

Рассмотрим, каким образом это можно сделать.

Используем функцию **implode**

Для объединения элементов одномерного индексного массива в строку используем стандартную функцию **implode()**.

В коде приведенного ниже листинга мы преобразовали одномерный **индексный массив** в строку и добавили полученную строку к ссылке.

example_4/index.php. Implode и индексный массив

```
<?php

// одномерный индексный массив
```

```

$auto = ["Toyota", "Camry", "2.5", "181", "Бензин"];

// преобразуем массив в строку
// в качестве разделителя строки используем символ /
$get = implode("/", $auto);

// тестовый вывод
// echo "Массив в строке: " . $get . "<p>";

// выведем внешнюю ссылку с параметрами
echo "<a href=target.php?auto=$get>Переход на новую страницу
с передачей GET-параметров</a>";

?>

```

example_4/target.php.

```

<?php

if(isset($_GET["auto"])){

    // преобразуем в массив принятый параметр

    $arr = explode("/", $_GET["auto"]);

    // выводим массив в браузер

    echo "<pre>";

    print_r($arr);

    echo "</pre>";

}

?>

```

Примечание. Обратите внимание, для демонстрационных примеров **example_4 / example_5** используется один принимающий файл **target.php**.

Использовать функцию **implode** для **ассоциативного массива не получится**. Если вы передадите ассоциативный массив в функцию **implode**, она объединит только **значения массива** и проигнорирует его ключи.

example_5. Implode и ассоциативный массив

```
<?php

// одномерный ассоциативный массив

$auto = ["brand" => "Toyota", "model" => "Camry", "engine" =>
"2.5", "power" => "181", "type" => "Бензин"];

// преобразуем массив в строку

// в качестве разделителя строки используем символ /

$get = implode("/", $auto);

// тестовый вывод

// echo "Массив в строке: " . $get . "<p>";

// выведем внешнюю ссылку с параметрами

echo "<a href=target.php?auto=$get>Переход на новую страницу
с передачей GET-параметров</a>";

?>
```

Вроде бы и получилось передать данные, но это явно не то, что я хотел.

Ассоциативный массив я создавал с целью получить пару: **ключ-значение**. Значит для сохранения пары необходимо использовать что-то другое.

Лучшим способом обработки значений массивов являются циклы.

Усложним задачу и попробуем сформировать строку запроса перебором элементов массивов, как говорят в таких случаях "на лету".

Используем составление строки запроса на лету

В предыдущей теме мы подробно рассмотрели вопросы **анонимных** функций и функций **обратного вызова**. Используем полученные знания для формирования строки соответствующего формата.

Выполним пример формирования строки запроса из **одномерного индексного массива**.

example_6/index.php. Составляем строку запроса из индексного массива

```
<?php

    // одномерный индексный массив

    $auto = ["Toyota", "Camry", "2.5", "181", "Бензин", "2011"];

    // используя функцию array_map перезапишем значения массива

    $auto = array_map(function ($value) {

        return "auto[]=" . $value;

    }, $auto);

    // объединим значения массива в строку

    $get = implode("&", $auto);

    // тестовый вывод

    // выведем сформированную строку для GET-параметров

    echo 'строка $get: ', $get , "<p>";

    // ссылка на target.php с передачей GET-параметров

    echo "<a href=target.php?$get>Переход на новую страницу с
    передачей GET-параметров</a>";

?>
```

example_6/target.php.

```
<?php

    if(isset($_GET["auto"])){

        // выводим массив из строки запроса в браузер

    }
```

```
        echo "<pre>";

        print_r($_GET["auto"]);

    }

?>
```

Примечание. Обратите внимание, в примерах **example_6 / example_7** используется один принимающий файл **target.php**. На стороне принимающего файла просто выводим полученные данные.

Сформируем строку запроса из **одномерного ассоциативного массива**.

example_7/index.php. Составляем строку запроса из ассоциативного массива

```
<?php

    // одномерный ассоциативный массив

    $auto = ["brand" => "Toyota", "model" => "Camry", "engine" =>
    "2.5", "power" => "181", "type" => "Бензин"];

    // объявление массива для сбора информации

    $arr = array();

    // перебираем массив, формируем значения нового массива

    foreach ($auto as $key => $value) {

        $arr[] = "auto[" . $key . "]= " . $value;

    }

    // объединим значения массива в строку

    $get = implode("&", $arr);

    // тестовый вывод

    // выведем сформированную строку для GET-параметров

    echo 'строка $get: ', $get , "<p>";
```

```
// ссылка на target.php с передачей GET-параметров

echo "<a href=target.php?$get>Переход на новую страницу с
передачей GET-параметров</a>";

?>
```

Отправка многомерных массивов

Передача данных одномерного массива в строке запроса, не самое сложное, что приходилось делать нам в этом курсе. Но, что если возникнет необходимость передать в сценарий данные **двумерного массива**. Например, такого:

```
<?php

$auto = [

    ["brand" => "Toyota", "model" => "Camry", "engine" =>
    "2.5", "power" => "181"],

    ["brand" => "Volkswagen", "model" => "Touareg",
    "engine" => "3.0", "power" => "249"]

];
```

В следующем примере сформируем строку запроса для передачи данных **двумерного массива**.

example_8/index.php. Пересылка многомерного массива

```
<?php

// двумерный ассоциативный массив

$auto = [

    ["brand" => "Toyota", "model" => "Camry", "engine" =>
    "2.5", "power" => "181"],

    ["brand" => "Volkswagen", "model" => "Touareg", "engine"
=> "3.0", "power" => "249"]

];
```

```

];

// объявим массив для сбора информации

$arr = array();

// запускаем цикл по внешнему индексному массиву

foreach ($auto as $index => $item) {

    // а затем по внутреннему ассоциативному массиву

    foreach ($item as $key => $val) {

        $arr[] = "auto[" . $index . "][" . $key . "]= " .
            $val;

    }

}

// формируем строку

$get = implode("&", $arr);

// выведем сформированную строку запроса

echo 'строка $get: ', $get , "<p>";

// выводим ссылку с сформированными в цикле GET-параметрами

echo "<a href='target.php?' . $get . '>Просмотреть
информацию об авто</a>";

?>

```

Обратите внимание на принимающий файл **example_11.php**, который должен присутствовать в вашей директории.

example_8/target.php. Файл, принимающий двумерный массив

```

<?php

if(isset($_GET["auto"])){

    // выводим массив из строки запроса в браузер

    echo "<pre>";

    print_r($_GET["auto"]);
}

```

```
echo "</pre>";

// образец доступа к элементам массива

echo $_GET["auto"][1]["brand"];

}

?>
```

Будьте внимательны. Формируемая строка оформляется нами в виде двумерного массива, но предопределенная переменная **\$_GET** сама является массивом. Таким образом, фактически, принимать будем **массив третьего уровня**.

Упаковка двумерного массива в строку соответствующего формата выполнена успешно. А если массив многомерный? Объем кода и сложность выполняемых манипуляций во вложенных циклах, естественно, возрастут.

Сложность формируемой строки ограничивается **фантазией** разработчика, способностью сервера принимать **большой объем** пересылаемых данных и **требованиями** реализуемого проекта.

Циклы - важная часть кодинга любого языка программирования. Но если можно заменить цикл одной единственно функцией, то почему бы этого не сделать.

Важно. Все становится гораздо проще, если использовать стандартный текстовый **JSON—формат** обмена данными.